Season 1 | Episode 6

Agentic Observability: The AI Architect's Essential Blueprint

Safe and Sound AI

Agentic Observability: The AI Ar...

PodBean

In this episode of Safe and Sound AI, we dive into the challenge of moving AI agents from impressive demos to robust, production-ready systems. We break down the principles of Agentic Observability, explaining how this essential "blueprint" provides the clarity needed to overcome the "black box" problem during both development and production.

Learn practical methods for monitoring key signals like tool usage and planning, discover how to diagnose the root causes of agent failures, and explore strategies for ensuring your agent delivers real-world value.

Transcript

[00:00:01] Welcome to Safe and Sound AI. Today we're pulling back the curtain a bit on, well, something really critical for AI's future, what it actually takes, you know, to

make AI agents work in the real world. Far beyond those, um, flashy demos we always see.

[00:00:17] Think about building a custom car. You get a great chassis, right. Drop in a super powerful engine and boom, you can make it roar right there in the garage. It sounds amazing. Looks incredible. That's your proof of concept.

[00:00:28] The demo that gets everyone buzzing.

[00:00:29] Exactly. But that amazing roar in the garage, that's not really a car, it's just a well, a noisy machine on wheels. The real challenge, and this is the part that often gets missed in all the excitement, is everything else.

[00:00:41] All the stuff that makes it a, you know, functional, safe vehicle. We're talking the transmission, the brakes, obviously steering, that really complex electrical system, and crucially, the central computer that needs to make all those different parts work together.

[00:00:54] That's what turns the demo into something real. And well think about a real car. To manage all that complexity to make sure everything's working like it should. You need diagnostics, constant feedback. You need to know what's happening under the hood.

[00:01:05] Right

[00:01:06] And for AI agents, that essential diagnostic tool, that way to really understand and manage these complex dynamic systems, that's what we call observability.

[00:01:15] So today we're doing a deep dive into the absolute essential dashboard. You need to, uh, really hit that top speed and keep it there for your AI agents.

[00:01:26] Agentic Observability.

[00:01:28] We're gonna explore exactly why this new kind of observability is well just so vital for AI agents

[00:01:34] Both in development and out in the wild.

[00:01:36] Exactly. Its role during those demanding development phases, and then it's unpredictable life in production. And then we will really drill down into the specific actionable insights you absolutely need to track to get your agents performing at their peak.

[00:01:50] And it's crucial because, you know, unlike traditional software or even say simply machine learning models, these AI agents operate with a level of, um, internal opacity. It can be incredibly frustrating,

[00:02:06] Like a black box sometimes.

[00:02:07] Very much so. It's like trying to diagnose a complex engine problem just by listening from the outside you hear something is happening, but you don't really know what or why, right?

[00:02:15] So this deep dive, it's really gonna reveal how you can gain true actionable clarity into these complex, often dynamic systems. Moving beyond just hearing the engine, you know, to truly understanding its every part.

[00:02:28] Okay, so let's jump right into that core problem then. If you've been working with AI agents, you've probably hit this wall.

[00:02:33] We're simply dumping all the raw data, all those traces and spans. It just creates this, this mountain of noise,

[00:02:39] Absolutely

[00:02:40] Overwhelming

[00:02:41] It is overwhelming and honestly not very useful for figuring out what your agent's actually doing. So what's the fundamental shift in approach we need here? Is it just about data volume or is it more about the quality and maybe the context of the data?

[00:02:53] That's a really good question because it's absolutely about context. I mean, what's truly necessary, what really helps is a single aggregated view of all the agents connected to an application.

[00:03:05] Okay. Like a central command center.

[00:03:06] Sort of, yeah. It gives you that crucial bird's eye view of the whole system.

[00:03:11] It lets a developer quickly zoom into the specific bits that actually need attention

[00:03:16] Rather than digging through everything.

[00:03:17] Exactly. But even more than that, it's vital to see the pathways of control, the agent follow, not just like a static list of steps it took, it's about understanding the emergent behavior that comes out of those pathways, and that's something you almost never see in traditional software in the same way.

[00:03:32] Right. To give you a really powerful visual for this, imagine, uh, like a weighted graph that represents your agent's decision making. Okay? The nodes are maybe the agent's, internal states or decision points, and the edges are the actions or the pathways it considers taking. Mm-hmm. Now picture the pathways the agent takes most often shown is these bright, thick lines, almost like a heat map on the graph.

[00:03:56] Yeah, I can see that

[00:03:57] This immediately shows you the agent's habits, right? Where it's spending its time. And crucially, it helps you spot if it's consistently going down some inefficient path or even an incorrect one, maybe due to some subtle bias it picked up, you could literally see the patterns emerge, the good and the bad.

[00:04:14] And honestly, I'd argue that observability during development. It's arguably even more critical for agents than it is for traditional ML or deep learning models. Really? Why is that? Well, because the developer often just cannot predict the initial outcome of an agent's run. It's inherently exploratory.

[00:04:33] You set it loose and see what happens.

[00:04:34] Pretty much you're not just training a static model, you're building something that reasons and acts in potentially unexpected ways. And this leads to multiple rapid iterations, even for what seems like a simple proof of concept, you're constantly experimenting, tweaking, you're essentially teaching it as you go.

[00:04:52] That's exactly where the rubber meets the road in development, isn't it? Those rapid iterations are essential. They let you catch that erroneous behavior early on, like those really frustrating hallucinations or when the agent just completely misuses a tool. It's given access to.

[00:05:08] Yeah. Tool misuse is a big one.

[00:05:09] It is, yeah. And since prompt engineering is, let's face it, the backbone of any agent.

[00:05:15] Yeah.

[00:05:15] Well, observability is the absolute key to doing it effectively. We've all seen how drastically prompts can alter agent behavior.

[00:05:23] Oh, completely. Night and day sometimes.

[00:05:25] Yeah. They're the agent's foundational instructions.

[00:05:27] Yeah.

[00:05:27] If they're not spot on, the agent will just, you know, go off the rails. I remember working on one agent. We thought we had a perfectly tuned in dev uhhuh, only to find out later. It was stuck in this infinite loop calling the same tool over and over because of some tiny parsing error. We missed. That kind of detailed observability, it saved us weeks of debugging, guesswork.

[00:05:46] That's such a classic example. Yeah, because when you're deep in prompt engineering, you're often running just tiny tweaks on a single prompt, right? Yeah. Many, many variations. Observability lets you actually compare and contrast those different prompt templates and see the resulting agent behavior side by side.

[00:06:02] It's kind of like A/B testing, but like. On steroids,

[00:06:05] Right? But much deeper,

[00:06:07] much deeper. Instead of just seeing a simple conversion rate, you're getting this deep insight into why one prompt leads to a more intelligent or, uh, more efficient outcome than another. Without that deep insight, you're basically just guessing.

[00:06:20] You're hoping your agent behaves rather than knowing why it does.

[00:06:23] And is that non-deterministic nature the key differentiator from standard ML tuning?

[00:06:28] I think so what makes it uniquely critical here is that non-determinism. A slight change in the prompt can lead to completely different, really complex chains of

thought and sequences of tool calls, and you need to be able to see all of that unfolding.

[00:06:41] Okay, so you've used these insights, you've built a solid agent in development, but the job's not over. Let's shift focus to when these agents are, you know, in the wild interacting with actual user. The main insight from production observability is it just about how the agent is being used or is there more to it?

[00:06:58] Maybe how it's behavior evolves or how users react to unexpected things? It does.

[00:07:03] That's such a crucial point. Users will inevitably query the system, interact with it in ways the developer never even dreamed of.

[00:07:11] Guaranteed

[00:07:12] Guaranteed. But here's the thing. This isn't a problem. In fact, it's a massive feature.

[00:07:16] It gives you this unvarnished data-driven path to building new functionality.

[00:07:22] Ah, so you learn from the unexpected usage.

[00:07:24] Exactly. It's like uncovering this unanticipated intelligence directly from your user base. It shows you exactly what features and improvements are truly needed based on what the majority of users are actually trying to do.

[00:07:34] Rather than just, you know, your internal assumptions or roadmap. It really unlocks that next level of agent capability.

[00:07:41] That makes a lot of sense. So extending an agent that's already live in production. Yeah. It's increasingly gonna be driven by these real world analytics.

[00:07:49] It has to be

[00:07:50] Which makes a robust observability tool just utterly essential.

[00:07:54] It lets your agent adapt and grow with its user base.

[00:07:57] Precisely. So what does this all mean for the specifics? Let's get practical. This is where the rubber meets the road. You know, where you actually build your agent's dashboard. Let's dive into the actual signals and patterns you should be monitoring to get that really granular insight.

[00:08:13] Okay. Sounds good. Let's start with a reflection. Now, this isn't just a simple output check, is it? It's often more complex like the agent generating a meta prompt to evaluate its own previous output against the original user intent.

[00:08:27] Yeah, exactly. A kind of recursive self-correction loop,

[00:08:30] Which sounds like it could get computationally expensive.

[00:08:32] If it happens too much,

[00:08:33] It definitely can. So the key metric here is the number of these reflection iterations. If you see a high number of these loops

[00:08:41] uhhuh,

[00:08:41] it often points to a deeper issue, like a fundamental mismatch in the agent's initial prompt. Or maybe its tool access isn't quite right. It suggests you need to reevaluate its core setup, not just tweak the edges,

[00:08:52] Right.

[00:08:53] It's like your car constantly rerunning its diagnostics because of some subtle engine knock. It might eventually get you there, but it's clearly struggling.

[00:09:01] Okay, moving on. Another critical area is tool usage. Your observability tool, it absolutely must be able to answer questions like: Which tools are genuinely easy versus say hard for the large language model to call successfully. Okay. Is the LLM generating incorrect tool names or arguments?

[00:09:19] Maybe because of some subtle misunderstanding buried in the prompt.

[00:09:22] That happens surprisingly often.

[00:09:23] It does. Yep. Is it calling the wrong tool for the job or maybe calling tools unnecessarily? Just wasting compute resources. And crucially, how well does it parse the results it gets back, and how well does it pass outputs between sequential tool calls?

[00:09:37] That's where that granular observability really shines. Failures here can be incredibly subtle. Like, uh, misinterpreting a successful API call as an error. It makes that detailed view critical.

[00:09:50] Got it. Okay. Next up, planning. So here you need to be constantly asking things like. Are there unnecessary steps in the agent's overall plan, or maybe missing steps,

[00:10:02] right?

[00:10:02] Is the plan itself flawed,

[00:10:04] and then is the agent actually following that plan faithfully, step by step? Or is it deviating in unexpected ways?

[00:10:10] Yeah, going off script,

[00:10:11] and are the right tools being used at the right steps and crucially, with the correct information being passed along? These questions seem vital to ensure your agent isn't just, you know, going rogue, wasting resources, or getting completely stuck.

[00:10:23] Exactly. Observing these planning patterns helps you debug not just what went wrong in the end, but where in the agent's reasoning process things went awry. It lets you fine tune its strategic thinking, not just its execution. Makes sense. And this brings up a really crucial point. For more complex setups for multi-agent systems, you absolutely need to observe the handoffs.

[00:10:44] Okay. What's specifically

[00:10:45] Well. Does the handoff between agent A and agent B actually capture all the relevant information needed? Or is something getting lost in translation and maybe even more basic, is it handing off to the right agent in the first place, or is there some misdirection happening?

[00:11:00] Ah, routing errors between agents

[00:11:02] Precisely and beyond these specific patterns.

[00:11:05] You also need to track basic technical limits. How often does the agent actually exceed its context window, which might cause it to, you know. Truncate information or just lose the threat of the conversation.

[00:11:16] Yeah. Context limits are a constant battle.

[00:11:18] They are. And how often does it trigger the guardrails you've put in place that indicates it's trying to do things it shouldn't. These aren't just like technical curiosities. They're really strong signals about the boundaries of your agent's current capabilities. They show you where it needs more refinement or maybe more explicit instructions.

[00:11:38] Okay. Let's, uh, let's try and unpack this and bring it all back together then.

[00:11:41] Mm-hmm.

[00:11:42] All these detailed low level metrics and patterns we've just discussed, they ultimately have to connect back to the high level business outcomes that actually matter, right?

[00:11:52] Absolutely. That's the end game.

[00:11:53] So what does this all mean for your bottom line? I mean, you need to be asking how often is the agent actually solving the underlying business problem that was built for?

[00:12:02] That's its fundamental effectiveness,

[00:12:04] right? Does it do the job?

[00:12:05] Then? Are my users happy? How often are they correcting the agent? Or you know, handholding it or just stepping in to take over completely 'cause the agent couldn't finish the task. That's your user satisfaction.

[00:12:16] Crucial metric, often overlooked

[00:12:18] And critically important.

[00:12:19] How much is this agent actually costing me per task, per user, per day? That's the real cost of operation you need to justify

[00:12:27] and ultimately that's what Agentic Observability provides. It's that critical feedback loop you need to answer these high level business critical questions. It's the only reliable way to measure, to continuously improve and to truly justify the value proposition of your AI agent.

[00:12:43] It's how you turn them from just a promising proof of concept into a robust production ready system that delivers real, tangible value.

[00:12:51] Well said. Thank you for joining us on this deep dive today. As you think about your own approach to understanding these complex AI systems, maybe consider what surprising insights you might uncover when you really start observing your agents in action.

[00:13:04] What might you find?

[00:13:05] Exactly. What hidden habits or unexpected pathways might they be taking? We really encourage you to reflect on what specific insights you would prioritize if you were building or managing an AI agent right now. And maybe what fundamental assumptions you hold about its behavior might get challenged once you truly start to see inside its operations. This podcast is brought to you by Fiddler AI. For more on observability , or more details on the concepts we discussed, see the article in the description.

Ready to try AI Observability?

Request demo

Product

AI Observability

Fiddler Trust Service

Agentic Observability

LLM Observability

ML Observability

Model Monitoring

Analytics

Pricing

Explore

Responsible AI

Build vs Buy

Governance, Risk Management, and Compliance

Government

Customer Experience

Lending and Trading

Lifetime Value

Company

About Us

Careers Hiring

Customers

Partners

Newsroom

Security

Contact Us

Resources

Blog

Docs

Resource Library

Webinars

Events

Support Center

Learn

Fiddler Guardrails Now Native to NVIDIA NeMo Guardrails — Industry's Fastest Guardrails

Fiddler Trust Service for LLM Scoring

Four Ways that Enterprises Deploy LLMs

AI Forward Summit

fiddler Product Solutions Pricing Resources Company Run free guardrails Reque

© 2025 Fiddler AI. All rights reserved.

Privacy Policy Terms of Use

Subscribe